

EFFICIENT PARALLEL JOB SCHEDULING USING GANG SERVICE*

Fabricio Alves Barbosa da Silva
LIP6, Dept ASIM
Universite Pierre et Marie Curie
4, Place Jussieu 75252 Paris Cedex 05
France
fabricio.silva@lip6.fr

and

Isaac D. Scherson
Department of Information and Computer Science
University of California, Irvine CA 92697-3425
USA
isaac@uci.edu

Received (received date)

Revised (revised date)

Communicated by Editor's name

ABSTRACT

Gang scheduling has been widely used as a practical solution to the dynamic parallel job scheduling problem. To overcome some of the limitations of traditional Gang scheduling algorithms, Concurrent Gang is proposed as a class of scheduling policies which allows the flexible and simultaneous scheduling of multiple parallel jobs. It hence improves the space sharing characteristics of Gang scheduling while preserving all other advantages. To provide a sound analysis of Concurrent Gang performance, a novel methodology based on the traditional concept of competitive ratio is also introduced. Dubbed dynamic competitive ratio, the new method is used to compare dynamic bin packing algorithms used in this paper. These packing algorithms apply to the Concurrent Gang scheduling of a workload generated by a statistical model. Moreover, dynamic competitive ratio is the figure of merit used to evaluate and compare packing strategies for job scheduling under multiple constraints. It will be shown that for the unidimensional case there is a small difference between the performance of best fit and first fit; first fit can hence be used without significant system degradation. For the multidimensional case, when memory is also considered, we concluded that the packing algorithm must try to balance the resource utilization in all dimensions simultaneously, instead of given priority to only one dimension of the problem.

Keywords: Parallel Job Scheduling, Gang scheduling, Coscheduling, Parallel Computation, Dynamic Algorithm Analysis, Competitive Analysis, Resource Management, Bin-Packing

*A preliminary version of this paper appeared in the 1999 International symposium on parallel architectures, algorithms and networks [28]

1. Introduction

Parallel job scheduling is an important problem whose solution may lead to better utilization of modern multiprocessors and/or parallel computers. The basic scheduling problem can be stated as follows: “Given the aggregate of all tasks of multiple jobs in a parallel system, find a spatial and temporal allocation to execute all tasks efficiently”. For the purposes of scheduling, we view a computer as a queueing system. An arriving job may wait for some time, receive the required service, and depart. The time associated with the waiting and service phases is a function of the scheduling algorithm and the workload. Some scheduling algorithms may require that a job wait in a queue until all of its number of required processors become available (as in variable partitioning [10]), while in others, like time slicing, the arriving job receives service immediately through a processor sharing discipline.

We focus on scheduling based on Gang Service [26], namely, a paradigm where all tasks of a job in the service stage are grouped into a Gang and concurrently scheduled in distinct processors. Given a job composed of N tasks, in Gang service these N tasks compose a process working set, and all tasks belonging to this process working set are scheduled simultaneously in different processors, i.e., Gang service algorithms is the class of algorithms that schedule on the basis of whole process working sets [26]. Reasons to consider Gang service are responsiveness [11], efficient sharing of resources [18] and ease of programming. In Gang service the tasks of a job are supplied with an environment that is very similar to a dedicated machine [18]. It is useful to any model of computation and any programming style. The use of time slicing allows performance to degrade gradually as load increases. Applications with fine-grain interactions benefit of large performance improvements over uncoordinated scheduling [14].

Gang service allows both the time sharing as well as the space sharing of the machine, and it was originally introduced by Ousterhout [26]. Performance benefits of Gang scheduling the set of tasks of a job has been extensively analyzed in [18, 11, 14, 36]. Packing schemes for Gang scheduling were analyzed in [9].

Some implementations of Gang service have been described in the literature. Hori et al. [17] describe a Gang scheduler implementation in a workstation cluster that allows the time sharing of a 64 processor machine interconnected by Myrinet. Feitelson and Rudolph proposed the distributed hierarchical control, which is a scalable implementation of Gang scheduling based on buddy systems. The distributed hierarchical control (DHC) [12, 15, 13] defines a control structure over the parallel machine and combines time-slicing with a buddy-system partitioning scheme. In [12] a DHC scheme for supporting Gang scheduling was proposed. Gang schedulers based on the distributed hierarchical control structure has been implemented for the IBM RS/6000 [16, 5], and its performance has been analyzed from a queueing theoretic perspective [33]. Suzuki and Walsh [34] proposed a variation of Gang scheduling for Fujitsu AP1000+ parallel computer dubbed Moderate coscheduling. This algorithm controls the order of priority of parallel processes managed by the local scheduler in processors which relaxes some of the strict conditions of Gang scheduling.

The problems in Gang scheduling are: Individual task blocking [10], performance of I/O bound and interactive jobs under Gang scheduling [20] and necessity of multi-context switch across the nodes of the processor, which causes difficulty in scaling [8].

In this paper we propose a class of scheduling policies, dubbed Concurrent gang, as a generalization of Gang service. It allows for the flexible simultaneous scheduling of multiple parallel jobs in a scalable manner. Also, a detailed analysis of resource sharing strategies for Concurrent Gang is made for one dimensional and multidimensional cases. This analysis is made through the definition of an average case analysis method for on-line algorithms with the definition of the dynamic competitive ratio. This analysis can be used, for instance, to compare packing strategies for Gang scheduling for a given workload model.

The architectural model we consider in this paper is a distributed memory multiprocessor with four main components: 1) Processor/memory modules (Processing Elements - PE), 2) An interconnection network that provides point to point communication, 3) A synchronizer, that synchronizes all components at regular intervals and 4) A front end, where jobs are submitted. This architecture model is very similar to the one defined in the BSP model [35]. We shall see that the synchronizer plays a important role in the scalability of Gang service algorithms.

Although it can be used with any programming model, Concurrent Gang is intended primarily to efficiently schedule SPMD jobs, which, in turn, is by far the most popular parallel programming style. We adopt here the SPMD programming model as the basis for workload generation to evaluate the efficiency of our proposed scheduling algorithms.

In section 2 some preliminary definitions are stated. Section 3 describes the Concurrent Gang algorithm. The one dimensional resource sharing problem for Gang service algorithms is stated and analyzed in section 4. The multi dimensional resource sharing problem is the subject of section 5.

2. Generalities and Definitions

Consider the scheduling of a set of parallel jobs. A useful tool to help visualize the time utilization in parallel machines is a two-dimensional diagram dubbed *trace diagram*. The trace diagram is also known in the literature as Ousterhout matrix [26]. Referring to figure 1, one dimension represents processors while the other dimension represents time. Through the trace diagram it is possible to visualize the time utilization of the set of processors given a scheduling algorithm.

Gang Scheduling can hence be defined with respect to the trace diagram as the Concurrent scheduling of the set of tasks of a job in a slice. Finding a scheduling becomes hence equivalent to computing the trace diagram for a given workload (a set of jobs). The trace diagram is first computed at power up and updated at each workload change (task arrival, completion, etc). Gang service algorithms are preemptive algorithms. We will be particularly interested in Gang service algorithms which are *periodic and preemptive*. Related to periodic preemptive algorithms are the concepts of cycle, slice, period and slot. A *Workload change* occurs at the ar-

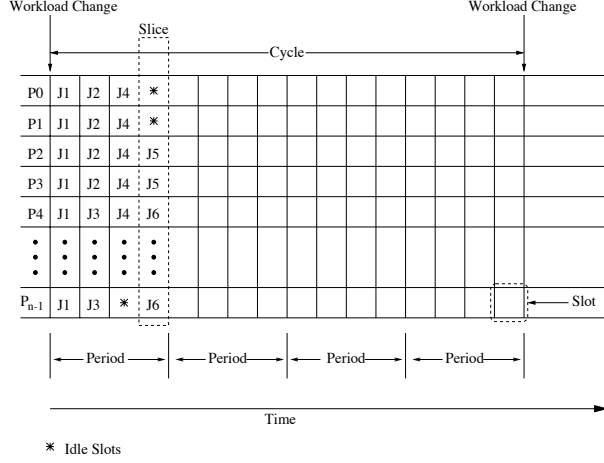


Fig. 1. Time Utilization in Parallel Machines

rival of a new job, the termination of an existing one, or through the variation of the number of eligible tasks of a job to be scheduled. The time between workload changes is defined as a *cycle*. Between workload changes, we may define a period that is a function of the workload and the spatial allocation. The period is the minimum interval of time where all jobs are scheduled at least once. A cycle/period is composed of *slices*; a slice corresponds to a time slice in a partition that includes all processors of the machine. A *slot* is the processors' view of a slice. A *Slice* is composed of N slots, for a machine with N processors. If a processor has no assigned task during its slot in a slice, then we have an idle slot. The number of idle slots in a period divided by the total number of slots in that period defines the *Idling Ratio*.

Some important results can be derived from the trace diagram:

Theorem 1 *Given a workload W composed of parallel SPMD jobs, for every temporal schedule S there exists a periodic schedule S_p such that the idling ratio of S_p is at most that of S .*

Proof. First, let's give a definition that will be useful in this proof. We define here *job happiness* in an interval of time as the number of slots allocated to a job divided by the total number of slots in the interval.

Define the progress of a job at a particular time as the number of slices granted to each of its tasks up to that time. Thus, if a job has V tasks, its progress at slice S may be represented by a *progress vector* of V components, where each component is an integer less than or equal to S . Observe that no task may lag behind another task of the same parallel SPMD job by more than a constant C number of slices. We call this behavior as *legal execution rule*. Note that C depends on the characteristics of the program. It can be determined, for instance, by global synchronization statements. In the worst case C slices corresponds to the completion time of the job. Observe that $C < \infty$, since the data partitions in a SPMD program are necessarily finite, so is the program itself. Therefore, no two elements in the progress vector can

differ by more than C . Define the differential progress of a job at a particular time as the number of slices by which each task leads the slowest task of the job. Thus a differential progress vector at time t is also a vector of V components, where each component is an integer less than or equal to C . The differential progress vector is obtained by subtracting out the minimum component of the progress vector from each component of the progress vector. The *system's differential progress vector* (SDPV) at time t is the concatenation of all job's differential progress vectors at time t . The key is to note that the SDPV can only assume a finite number of values. Therefore there exists an infinite sequence of times $t_{i_1}, t_{i_2}, \dots$ such that the SDPVs at these times are identical.

Consider any time interval $[t_{i_k}, t_{i'_k}]$. One may construct a periodic schedule by cutting out the portion of the trace diagram between t_{i_k} and $t_{i'_k}$ and replicating it indefinitely along the time axis.

We claim that such a periodic schedule is legal. From the equality of the SPDVs at t_{i_k} and $t_{i'_k}$ it follows that all tasks belonging to the same job receive the same number of slices during each period. In other words, at the end of each period, all the tasks belonging to the same job have made equal progress. Therefore, no two tasks lag behind another task of the same job by more than a constant number of slices.

Secondly, observe that it is possible to choose a time interval $[t_{i_k}, t_{i'_k}]$ such that the happiness of each job during this interval is at least as much as in the complete trace diagram. This implies that the happiness of each job in the constructed periodic schedule is larger than or equal to the happiness of each job in the original temporal schedule.

Therefore, the idling ratio of the constructed periodic schedule must be less than or equal to the idling ratio of the original temporal schedule. Since the fraction of area in the trace diagram covered by each job increases, the fraction covered by the idle slots must necessarily decrease. This concludes the proof. $\square$

A consequence of the previous theorem is stated in the following corollary:

Corollary 1 *Given a Workload W , for the set of all feasible periodic schedules S , the schedule with smaller idling ratio is the one with smaller period.*

Proof. The feasible schedule with smaller period is the one which has the smaller number of slices (resulting in a smaller number of total slots) and which packs all jobs as defined in the Concurrent Gang algorithm. The number of occupied slots is the same for all feasible periodic schedules, since the workload is the same. So the ratio between the number of idle slots, which is the difference between the total number of slots and the number of occupied slots, and the total number of slots is minimized when we have a minimum number of total slots, which is the case in the minimum period schedule. $\square$

2.1. Other Definitions

We will also consider *clairvoyant scheduling algorithms* as those that may use knowledge of the jobs' execution times to assign time intervals to jobs in a set of processors. *Non clairvoyant Scheduling Algorithms* assign time interval to jobs in

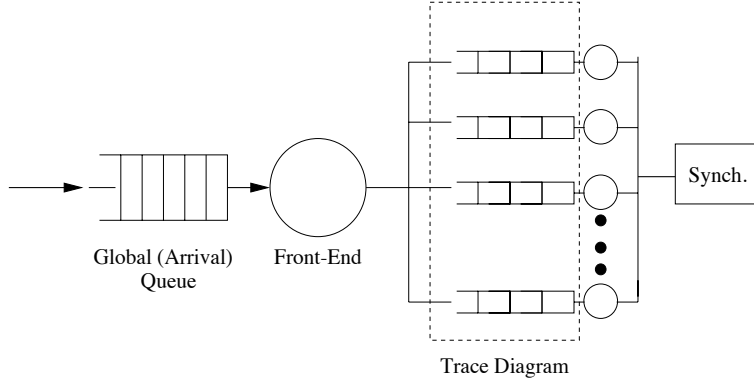


Fig. 2. Modeling Concurrent Gang class algorithms

a set of processors without knowing the execution times of jobs that have not yet been completed. A scheduling problem is said to be *static* if all release times are 0, i.e., all the jobs are available for execution at the start of the schedule. A *dynamic* scheduling problem allows arbitrary (nonnegative) release jobs. A *Parallel* job is composed of tasks, and we define *preemptive scheduling algorithms* as those where the execution of any task can be suspended at any time and resumed later from the point of preemption. We define *degree of parallelism* of job J as the maximum number of tasks of job J .

3. Concurrent Gang

In this section we present the Concurrent Gang algorithm, which is an evolution of the traditional Gang scheduler in the sense that it solves the problem related to task blocking in Gang scheduling, and also gives better service to I/O bound and interactive jobs. Beyond that, Concurrent Gang keeps all advantages of Gang Scheduling.

3.1. Definition of Concurrent Gang

Referring to figure 2, the architectural model of the machine includes four main components: 1) Processor/memory modules (Processing Elements - PE), 2) An interconnection network that provides point to point communication, 3) A synchronizer, that synchronizes all components at regular intervals of L time units and 4) A front end, which is a host computer where jobs are submitted. This architecture model is similar to the one defined in the BSP model [35]. For the definition of Concurrent Gang we view the parallel machine as composed of a general queue of jobs to be scheduled and a number of servers, each server corresponds to one processor. Each processor may have a set of tasks to execute. Scheduling actions are made at two levels: in the case of a workload change, global spatial allocation decisions are made in the front end scheduler, which stores the trace diagram and decides in which portion of it a job will run. The front end will allocate the jobs in the global queue depending on a predefined packing strategy, always trying to

minimize the number of slices used. Observe that, depending on the packing strategy, the trace diagram can be completely recomputed at each workload change or simply updated. The switching of local tasks in a processor as defined in the trace diagram is made through local schedulers, independently of the front end.

A local scheduler in Concurrent Gang is composed of two main parts: a local Gang scheduler module and a standard local task scheduler. The Gang Scheduler module schedules the next task indicated in the trace diagram at the arrival of a synchronization signal. The local task scheduler is responsible for scheduling specific tasks (as described in the next paragraph) allocated to a PE that do not need global coordination and it is similar to a UNIX scheduler. The Gang Scheduler has precedence over the local task scheduler.

We may consider two types of tasks in a Concurrent Gang scheduler: Those that should be scheduled as a Gang with other tasks in other processors and those for which Gang scheduling is not mandatory. Examples of the first class are tasks that compose a job with fine grain synchronization interactions [14] and communication intensive jobs [8]. Examples for the second class are local tasks or tasks that compose an I/O bound parallel job, for instance. In [20] Lee et al. proved that response time of I/O bound jobs suffers under Gang scheduling and that may lead to significant CPU fragmentation. On the other hand a traditional UNIX scheduler does a good job at scheduling I/O bound tasks since it gives high priority to I/O blocked tasks when the data becomes available from disk. As those tasks typically run for a small amount of time and then block again, giving them high priority means running the task that will take the least amount of time before blocking, which is coherent with the theory of uniprocessors scheduling where the best scheduling strategy possible under total completion time is Shortest Job First [24]. In the local task scheduler of Concurrent Gang, such high priority is preserved. Another example of jobs where Gang scheduling is not mandatory are embarrassingly parallel jobs. As the number of iterations among tasks belonging to this class of jobs are small, the basic requirement for scheduling an embarrassingly parallel job is to give those jobs the larger fraction of CPU time possible, even in an uncoordinated manner.

In Concurrent Gang, each PE classifies each one of its allocated tasks into classes. Examples of such classes are: I/O intensive, Synchronization intensive, and Computation intensive. Each one of these classes is similar to a fuzzy set [37]. A fuzzy set associated with a class A is characterized by a membership function $f_A(x)$ which associates each task T to a real number in the interval $[0,1]$, with the value of $f_A(T)$ representing the “degree of membership” of T in A. Thus, the nearer the value of $f_A(T)$ to unity, the higher the degree of membership of T in A. For instance, consider the class of I/O intensive tasks, with its respective characteristic function $f_{IO}(T)$. A value of $f_{IO}(T) = 1$ indicates that task T has executed only I/O statements, while a value of $f_{IO}(T) = 0$ indicates that task T has executed no I/O statement at all.

The actual number of classes depends on the architecture of the machine. The degree of membership of each local task to each class is computed based on the number of statements of each class that occurs in the execution of the program.

The local task scheduler defines a priority for each task allocated to the corresponding PE. The priority of each task is defined based on the degree of membership of a task to each one of the classes defined in function of the architecture. Formally, the priority of a task T in a PE is defined as:

$$Pr(T) = \max(\alpha \times f_{IO}, f_{COMP}) \quad (1)$$

Where f_{IO}, f_{COMP} are the degree of membership of task T to the classes I/O intensive and computation intensive. The choices made in equation 1 intend to give high priority to I/O intensive and computation intensive jobs, since such jobs can benefit the most from uncoordinated scheduling. The multiplication factor α for the class of I/O intensive jobs gives higher priority to I/O bound tasks over computation intensive tasks, since those jobs have a higher probability to block when scheduled than computing bound tasks (observe that $\alpha > 1$). On the other hand, synchronization intensive and communication intensive jobs have low priority since they require coordinated scheduling to achieve efficient execution and machine utilization [14, 8]. A synchronization intensive or communication intensive phase will reflect negatively over the degree of membership of the class computation intensive, reducing the possibility of a task be scheduled by the local task scheduler. Among a set of tasks of the same priority, the local task scheduler uses a round robin strategy.

In practice, the operation of the Concurrent Gang scheduler at each processor will proceed as follows: The reception of the global synchronization signal will generate an interrupt that will make each processing element schedule tasks as defined in the trace diagram. If a task blocks, control will be passed to one of the other tasks allocated in the PE. The chosen task is defined by the local task scheduler of the PE as a function of the priority assigned to each one of the tasks, and it will be the one with higher priority.

In the event of a job arrival, a job termination or a job changing its number of eligible tasks the front end Concurrent Gang Scheduler will :

- 1 - Update Eligible task list
- 2 - Allocate Tasks of First Job in General Queue.
- 3 - While not end of Job Queue
 - Allocate all tasks of remaining parallel jobs
 - using a defined spatial sharing strategy
- 4 - Run

Between Workload Changes

- If a task blocks or in the case of an idle slot, the local task scheduler is activated, and it will decide to schedule a new task based on:

- Availability of the task (task ready)
- Priority of the task defined by the local task scheduler.

For rigid jobs, i.e. jobs where the number of tasks does not change, the relevant events which define a workload change are job arrival and job completion.

All processors change context at same time due to the signal coming from a central synchronizer. The local queue positions represents slots in the scheduling trace diagram. The local queue length is the same for all processors and is equal to the number of slices in a period of the schedule. It is worth noting that in the case of a workload change, only the PEs concerned by the modification in the trace diagram are notified.

In the case of creation of a new task by a parallel task, or parallel task completion, it is up to the local scheduler to inform the front end of the workload change. The front end will then take the appropriate actions depending on the predefined space sharing strategy.

Scalability of the Concurrent Gang algorithm is improved due to the presence of the synchronizer working as a global clock, which allows the scheduler to be distributed among all processors. The front end is only activated in the event of a workload change, and decisions in the front end are made as a function of the chosen space sharing strategy.

Concurrent Gang is a strategy that increases utilization and throughput in parallel machines as compared to other implementations of Gang service, for the same resource sharing strategy, as simulation studies indicate [29, 31, 30]. The Processor utilization in Concurrent Gang is improved because, in the event of an idle slot, Concurrent Gang always tries to schedule other tasks that are either local tasks (although local tasks are not considered in the simulations) or tasks that do not require, at that time, coordinated scheduling with other tasks of the same job. This is the case, for instance, of I/O intensive tasks and Computation intensive tasks.

A question that remains is how to improve resource sharing in Concurrent Gang as a function of one or multiple constraints in order to maximize simultaneously the utilization of different resources such as number of PEs, memory, disk capacity, etc. This analysis is the subject of the following sections.

4. Resource Sharing in Concurrent Gang: One Dimensional case

In the one dimensional resource sharing problem, jobs are represented by only one parameter: the number of tasks (which is equal to the number of required processors in Concurrent Gang) that compose the job. The machine is characterized by the number of processors. We suppose that other resources, such as memory, have infinite availability. We will analyze in this section the dynamic variation, where arrival times can be different from zero. Observe that this problem is similar to the one dimensional dynamic (on-line) bin-packing problem as will be shown below.

4.1. Packing in Concurrent Gang

Recall that the computation of a schedule (i.e. the computation of the trace diagram) can be reduced to a bin packing problem. In the classical, one dimensional bin-packing problem, a given list of items $L = I_1, I_2, I_3, \dots$ is to be partitioned into

a minimum number of subsets such that the items on each subset sum to no more than B , which is the capacity of the bins. In the standard physical interpretation, we view the items of a subset as having been packed in a bin of capacity B . This problem is NP-Hard [2], so research has concentrated on algorithms that are merely close to optimal. For a given list L , let $OPT(L)$ be the number of bins used in optimal packing, and define:

$$s(L) = \lceil \frac{\sum |I_j|}{B} \rceil \quad (2)$$

Note that for all lists L , $s(L) \leq OPT(L)$. For a given algorithm A , let $A(L)$ denote the number of bins used when L is packed by A and define the waste $w_A(L)$ to be $A(L) - s(L)$.

When applying bin-packing to parallel job scheduling, bins corresponds to slices in the trace diagram, and items represents SPMD jobs.

In this paper we deal with bin-packing algorithms that are dynamic (also dubbed “on-line”). A bin packing algorithm is dynamic if it assigns items to bins in order $(I_1, I_2, \dots)$, with item I_i assigned solely on the basis of the sizes of the preceding items and the bins which they are assigned to, without reference to the size or number of remaining items [27, 2]. Two of the most well known strategies for dynamic bin-packing are first fit and best fit. In first fit, the next item to be packed is assigned to the lowest-indexed bin having an unused capacity no less than the size of the item. In best fit the used bins are sorted according to their capacities. The item to be packed is assigned to the bin with the smallest capacity that is sufficient. Best fit can be implemented to run in time $O(N \log N)$, and among online algorithms offers perhaps the best balance between worst and average case packing performance [2]. For instance, the only known on-line algorithm with better expected waste than best fit is the considerably more complicated algorithm of [27] that has expected waste $\Theta(N^{1/2} \log^{1/2} N)$ (compared with $\Theta(N^{1/2} \log^{3/4} N)$ for best fit) which is the best possible for any dynamic algorithm; this algorithm however has an unbounded asymptotic worst case-ratio.

However, it should be stressed that the problem that we consider in this paper is slightly different from the original dynamic bin packing problem, since each item has a duration associated with it. As items represent SPMD jobs, the duration represent the time it takes to run on a dedicated machine.

4.2. Dynamic Competitive Ratio

Competitive ratio(CR) based metrics [3, 24] are used to compare various space sharing strategies. The reason is that the competitive ratio is a formal way of evaluating algorithms that are limited in some way (e.g., limited information, computational power, number of preemptions) [3]. This measure was first introduced in the study of a memory management problem [23, 32]. The Competitive ratio [19, 3] for a scheduling algorithm A is defined as:

$$CR(n) = \sup_{J: |J|=n} \frac{A(J)}{OPT(J)} \quad (3)$$

Where $A(J)$ denotes the cost of the schedule produced by algorithm A, and $OPT(J)$ denotes the cost of an optimal scheduler, all under a predefined metric M. One way to interpret the competitive ratio is as the payoff to a game played between an algorithm A and an all-powerful malevolent adversary OPT that specifies the input J [19].

We are interested in the dynamic case, where we have a sequence of jobs $J = \{J_1, J_2, J_3, J_4, \dots\}$, with an arrival times $a_i \geq 0$ associated with each job, which is the case for jobs submitted to parallel supercomputers, as several workload studies show [9, 7]. Observe that consecutive arrival times can vary between seconds to hours, depending on the hour of the day [9]. For instance, let's consider a machine that implements a Gang scheduler using the trace diagram (an example is [25]). Upon arrival of a new job, the front end will look for the first slice with sufficient number of processors in the trace diagram (which is stored in the front end), will allocate the incoming job on that slice, will update the trace diagram, and the new job will start running in the next period. The same sequence of actions is taken for subsequent jobs.

For the dynamic case as defined in the previous paragraph, the definition of equation 3 is not convenient. For a dynamic scheduling the number of jobs n can be of order of thousands or tens of thousands of jobs, but they are *spaced in time*, in a way that, at each instant of time, we would have typically tens of jobs at most scheduled in the machine. Beyond that, competitive analysis has been criticized because it often yields ratios that are unrealistic high for "normal" inputs, since it consider the worst case workload, and as a result it can fail to identify the class of online algorithms that work well [19]. These facts led us to propose a new methodology for comparing dynamic algorithms on parallel scheduling based on the competitive ratio.

For the application of CR methodology in dynamic scheduling, let's consider as reference (adversary) algorithm the optimal algorithm OPT for a predefined metric M applied at each new arrival time. The OPT scheduler will be a clairvoyant dynamic adversary, with knowledge about all arrival times and the characteristics of all jobs. We will call this methodology of comparing dynamic algorithms as Dynamic Competitive Ratio (CR_d), and the scheduler defined by applying OPT at arrival times as OPT_d . Formally we have:

$$CR_d(N) = \frac{1}{N} \sum_{\tau=1}^N \frac{A(\tau)}{OPT_d(\tau)} \quad (4)$$

Where N represents the number of arrival times considered. Observe that CR_d only varies at arrival times. As Workload we have a (possibly infinite) sequence of jobs $J = \{J_1, J_2, J_3, J_4, \dots\}$, with an arrival time $a_i \geq 0$ associated with each job. At the time of each arrival, workload changes are taken into consideration in a way

that only those jobs that are still running at the arrival time are considered by both A and OPT algorithms.

A very important question is the determination of the workload to be used in conjunction with CR_d to compare different algorithms. When choosing the new coming job at each arrival time, we have two possibilities : either selecting a “worst case” job that would maximize the CR_d or considering synthetic workload models with arrival times and running times being modeled as random variables. Since with the “worst case” option we may create fake workloads that never happen in practice and leads to the sort of criticism we cited before, we believe that the best option is to use one of the synthetic workload models that have been recently proposed in the literature [7, 9]. These models and its parameters have been abstracted through careful analysis of real workload data from production machines. The objective with this approach is to produce an average case analysis of algorithms based on real distributions.

A lower bound for the packing problem under dynamic competitive ratio is derived in the following theorem.

Theorem 2 *For the dynamic packing of SPMD jobs in the trace diagram and for any non-clairvoyant scheduler, $CR_d(N) \geq 1$, $N > 0$*

Proof. Consider $N=1$. In the case of a workload composed by one embarrassingly parallel job with a degree of parallelism P , if we have a non clairvoyant scheduler that schedules each task in a different processor, as would do OPT_d , we have $CR_d(1) = 1$.

Conversely, an optimal clairvoyant scheduler will always be capable of producing a scheduling at least as good as a non-clairvoyant one, since the clairvoyant scheduler has all the information available about the workload at any instant in time. So $CR_d(N) \geq 1$ $\square$

For bin-packing, the reference or optimal algorithms will be simply the sum of item sizes $s(L)$, since $s(L) \leq OPT(L)$, and it can be easily computed. However, in order to use CR_d to compare the performance of algorithms, we must first define precisely the workload model we will use in the CR_d computation, which is done in the next section.

4.3. Workload Model

The workload model that we consider in this paper was proposed in [7]. This is a statistical model of the workload observed on a 322-node partition of the Cornell Theory Center SP2 from June 25, 1996 to September 12, 1996, and it is intended to model rigid job behavior. During this period, 17440 jobs were executed.

The model is based on finding Hyper-Erlang distributions of common order that match the first three moments of the observed distributions. Such distributions are characterized by 4 parameters:

- $\mathbf{p}$ – the probability of selecting the first branch of the distribution. The second branch is selected with probability $1 - p$.
- λ_1 – the constant in the exponential distribution that forms each stage of the first branch.

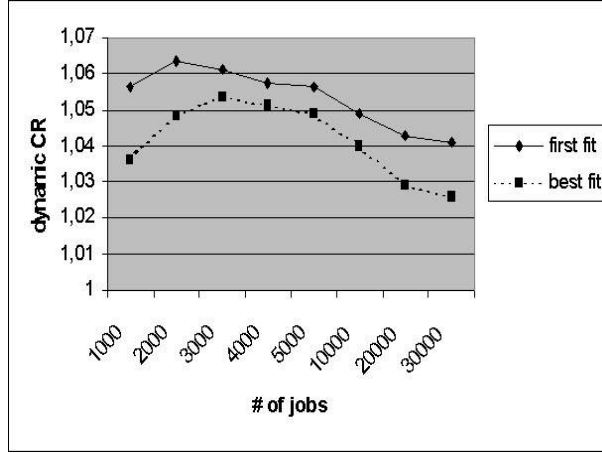


Fig. 3. Sequence 1 - no migration

- λ_2 – the constant in the exponential distribution that forms each stage of the second branch.

- $\mathbf{n}$ – the number of stages, which is the same in both branches.

As the characteristics of jobs with different degrees of parallelism differ, the full range of degrees of parallelism is first divided into subranges. This is done based on powers of two. A separate model of the inter arrival times and the service times (runtimes) is found for each range. The defined ranges are 1, 2, 3-4, 5-8, 9-16, 17-32, 33-64, 65-128, 129-256 and 257-322.

Tables with all the parameter values are available in [7].

4.4. CR_d applied to First Fit and Best Fit

We conducted experiments measuring the CR_d of first fit and best fit strategies. Our objective is to verify the behavior of these on-line packing strategies in terms of number of slices used when compared against a clairvoyant on-line algorithm. In particular, we are interested in knowing if the non-optimality of either first fit or best fit can lead to an unbounded growth of the number of slices in a period when compared against the optimal on-line algorithm for the workload model chosen. The experiments consisted of computing the CR_d of both best fit and first fit for a sequence of SPMD jobs generated through the model described in the previous section. The machine considered has 128 processing elements, and the size of jobs varied between 1 and 128 tasks, divided in 8 ranges. Then the first and best fit strategies were applied, and the number of slices used by each algorithm for each job arrival was computed. This number was then divided by the number of slices that would be used considering the sum $s(L)$ as defined in equation 2. The smaller the number of slices, the smaller the period is, with more slots dedicated to each job over time. Two cases were simulated:

- With job migration - All jobs return to the central queue and are redistributed among all processors at each new arrival.

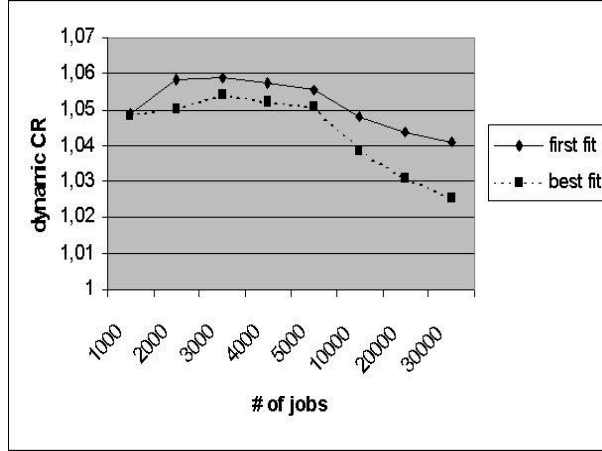


Fig. 4. Sequence 2 - no migration

- Without job migration - In this case an arriving job is allocated accordingly to a given algorithm without changing the placement of other jobs.

Two sequences of jobs were randomly generated using the model described in the previous subsection. Both sequences have 30000 jobs. Results for the first sequence without task migration are shown in figure 3. The horizontal axis represents number of job arrivals. We can see that best fit always achieved better results than first fit, but with a small difference between the two strategies. The bigger difference between the CR_d of first fit and the CR_d of best fit was around 2%. Results for the second sequence of jobs are shown on figure 4. The results are similar to the one obtained for the first figure, but the larger difference for this sequence was also around 2%. Observe that for a large number of jobs (> 10000) results of both sequences are almost equal for both the best fit and first fit algorithms. There is no unbounded growth of the number of slices when compared against the clairvoyant on-line algorithm in any case.

CR_d calculation for both sequences when task migration is considered are shown in figures 5 and 6. We can observe that the CR_d for both algorithms is at least one order of magnitude smaller than the results with no migration. As a consequence, the difference of the CR_d between the two algorithms become even smaller, with less than 1% in the worst case. Again, CR_d calculations for the two sequences became almost equal for a large number of jobs.

5. Resource Sharing in Concurrent Gang: Multi-dimensional case

In the multi-dimensional case the resources available in a machine are represented by a m-dimensional vector $R = (R_1, R_2, \dots, R_m)$ and the resources required by a job J are represented by a k-dimensional vector $J = (J_1, J_2, \dots, J_k), k \leq m$. Of particular interest for Gang service algorithms is the amount of memory required for each job, since most of the parallel machines available today do not have support for virtual memory, and the limited amount memory available determine the

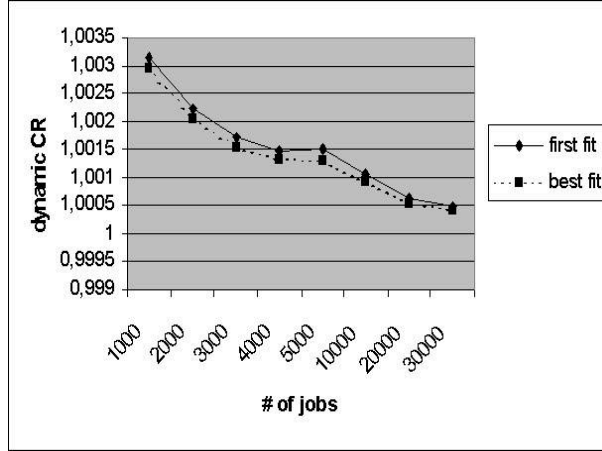


Fig. 5. Sequence 1 - with migration

number of jobs that can share a machine at any given time.

Maximizing memory utilization in order to allow multiple jobs to be scheduled simultaneously is a critical issue for Gang service systems. Many parallel applications demand a large amount of memory to run, and since these machines normally do not have support for virtual memory, eventually all applications submitted to a machine at a given time will not fit into the main memory available, which means that some jobs will have to wait before receiving service.

In the case of a distributed memory machine, the machine itself is modeled as a p -dimensional vector $R(t) = (R_1, R_2, \dots, R_p)$ where R_i represents the amount of memory available at time t in node i and p is the number of processors of the machine. The job is represented as a k -dimensional vector $J(t) = (J_1, J_2, \dots, J_k)$, where k is the maximum number of tasks of a job and J_i is the amount of memory required for task i at time t . As we are dealing with SPMD jobs, this formulation can be simplified as we will consider that the amount required by all tasks will be the same, so the requirements of a job are represented by a two-dimensional vector $J = (P_J, M_J)$, where P_J is the number of tasks (which is equal to number of processors in Gang service algorithms) of job J and M_J is the maximum amount of memory required by each task, which will be the same for all tasks.

Observe that this problem is different from the two-dimensional (geometric) bin-packing problem [1], in which rectangles are to be packed into a fixed width strip so as to minimize the weight of packing, since the memory segments required by a job do not need to be contiguous. It also differs from the vector bin packing problem. In the d -dimensional version of the vector packing problem, the size of an item is a vector $I = (I_1, I_2, \dots, I_d)$ and the capacity of a bin is a vector $C = (C_1, C_2, \dots, C_d)$. No bin is allowed to contain items whose vector sum exceeds C in any component. In the multidimensional resource sharing problem stated in this section the dimensions of the size of an item and the capacity of a bin can be different, as we are considering a distributed memory machine and the resources

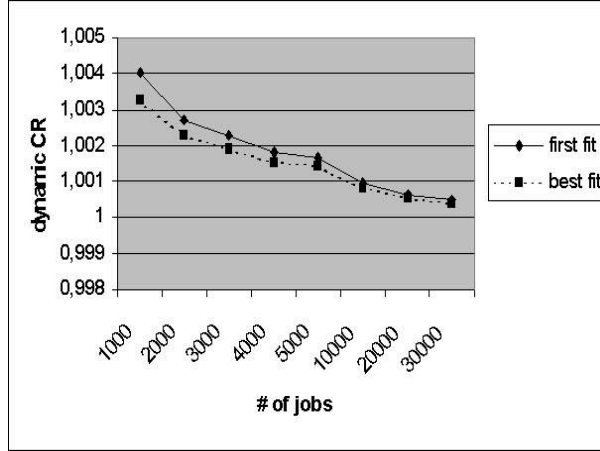


Fig. 6. Sequence 2 - with migration

(memory) available in the machine are defined in a per-processor basis, not in a global basis. This leads to different solutions than those proposed in [22], since a direct association between i th component of the resource vector R and the i th component of requirement vector J is not mandatory.

5.1. Memory Fit algorithm

Our objective is to maximize the number of jobs that can be allocated in the same period. To do so, the packing strategy must take into account the amount of memory available on each node. That is the objective of the *memory fit algorithm* proposed in this section. In the memory fit algorithm, the front end at each job arrival chooses the slice and the processors that will receive an incoming job as a function of the amount of memory available, in order to balance the usage of memory among the nodes of the machine. If more than one solution is possible, the front end chooses a slice using the best fit strategy. If there is no set of processors in the available slices with sufficient memory to receive the new job, the front end can create a new slice to accommodate the new coming job. Of course a new slice can also be created due to insufficient number of processors in the existing slices, as in best fit and first fit.

If a job arrives and there is no sufficient resources available for its execution, it waits in the global queue until the amount of memory required by the job becomes available. However, if another jobs arrives later and there is a sufficient amount of memory to schedule the job, it is scheduled immediately, as in the backfilling strategy.

When choosing a slice for scheduling a new coming job, first the memory fit chooses the k processors in a slice where there is more memory available, where k is the degree of parallelism of a job. Then computes a "measure of balance" in order to know how much that particular allocation reduces imbalance. The algorithm repeats this sequence for every existing slice and also considers the possible creation

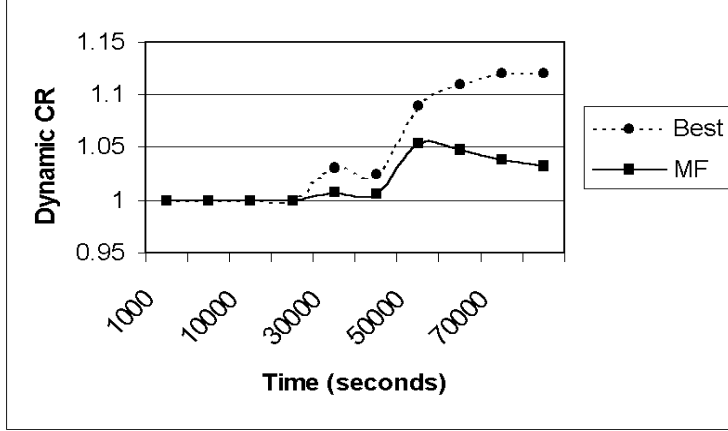


Fig. 7. Dynamic competitive ratio applied to memory fit

of a new slice. The solution that minimizes the memory imbalance in the machine is the one chosen.

For the measure of memory imbalance, in this paper we have chosen a *max/average* balance measure [21]. For a given slice, the front end considers the allocation of a job in the set of processors where there is more memory available, and then compute the following measure:

$$B(slice) = \frac{\max_i M_i}{\sum_{i=1}^P M_i} \quad (5)$$

Where M_i is the amount of memory available in node i . A lower value of B indicates a better balance.

5.2. CR_d applied to Memory fit

In this section we apply a sequence of jobs to evaluate the performance under CR_d of the memory fit packing algorithm and we compare with the best fit algorithm of section 4, with no migration in both cases. This algorithm was modified to take into account memory requirements. The bin chosen is the one that minimizes processor waste and at the same time has sufficient memory to accommodate the job, regardless of any memory balance. However, we must first define a memory usage for each job in the sequence, since the workload model used did not take into account memory requirements. In this paper, we considered that the memory usage of a job has a direct correlation with this size. This assumption is coherent with workload modeling that was used as a guide line for scheduler design in parallel machines, notably in the Tera MTA [6]. In this work was observed that jobs with large amounts of parallelism have large memory requirements and use a lot of resources. Small tasks, on the other hand, use resources in bursts, have small memory requirements and are not very parallel. Based on these observations we defined that for each job, the memory utilization of a job varied between 2 MB and 256 MB per node in

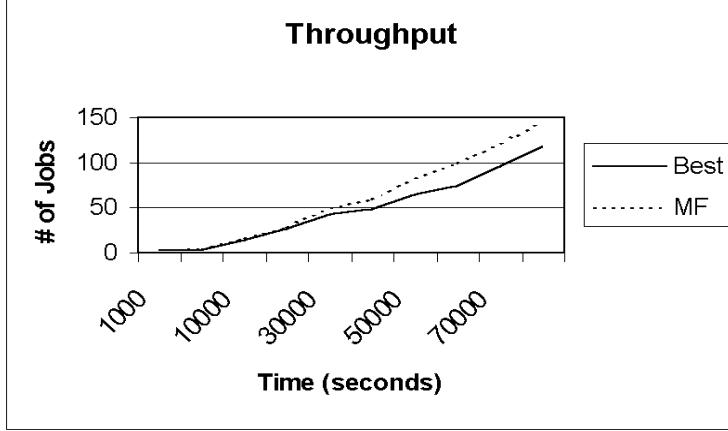


Fig. 8. Throughput of both best fit and memory fit

function of the size of the job and the ranges defined in subsection 4.3. For instance, we considered that 1 task jobs require 2 MB of memory, 2 task jobs require 4 MB per task, 3-4 task jobs require 8 MB per task, and so on. Each node had 512 MB of main memory.

Our objective is to maximize the number of jobs that fit in one period of the machine by maximizing the memory utilization of the machine. The reference algorithm always has a memory utilization of 100%, given that enough jobs have arrived to fill in the memory, due to its clairvoyance. Simulation results are shown in figures 7 and 8. Figure 7 illustrates the evolution of CR_d over time under memory utilization. Figure 8 shows the throughput of both algorithms. Observe that the number of arrivals and arrival times submitted to both algorithms are the same, since the same sequence of jobs were submitted to both algorithms. In both cases the horizontal axis represents time in seconds. The evolution of the system was simulated for 10000, 20000, 30000, 40000 and 50000 seconds. These values are chosen in order to verify the evolution of the system during a working day (50000 seconds represents 13.8 hours).

We can observe that the Memory Fit algorithm not only yields better memory utilization than best fit, but it also improves the throughput, as illustrated by figure 8. This is a direct consequence of the capability of the memory fit algorithm of allocating more jobs in the same period.

Acknowledgments

The first author is supported by Capes, Brazilian Government, grant number 1897/95-11. The second author is supported in part by the Irvine Research Unit in Advanced Computing and NASA under grant #NAG5-3692.

References

1. E.G. Coffman, M.R. Garey, and D.S. Johnson. Bin Packing with Divisible Item

- sizes. *Journal of Complexity*, 3:406–428, 1987.
2. E.G. Coffman, D.S. Johnson, P.W. Shor, and R.R. Weber. Markov Chains, Computer Proofs, and Average Case Analysis of Best Fit Bin Packing. In *Proceedings of the 29th ACM Symposium on Theory of Computing*, pages 412–421, 1993.
 3. J. Edmonds, D.D. Chinn, T. Brecht, and X. Deng. Non-Clairvoyant Multiprocessor Scheduling of Jobs with Changing Execution Characteristics (extended abstract). In *Proceedings of the 1997 ACM Symposium of Theory of Computing*, pages 120–129, 1997.
 4. A. Hori et al. Implementation of Gang Scheduling on Workstation Cluster. *Job Scheduling Strategies for Parallel Processing*, LNCS 1162:126–139, 1996.
 5. F. Wang et al. A Gang Scheduling Design for Multiprogrammed Parallel Computing Environments. *Job Scheduling Strategies for Parallel Processing*, LNCS 1162:111–125, 1996.
 6. G. Averson et al. Scheduling on the Tera MTA. *Job Scheduling Strategies for Parallel Processing*, LNCS 949, 1995.
 7. J. Jann et al. Modeling of Workloads in MPP. *Job Scheduling Strategies for Parallel Processing*, LNCS 1291:95–116, 1997.
 8. Patrick G. Solbalvarro et al. Dynamic Coscheduling on Workstation Clusters. *Job Scheduling Strategies for Parallel Processing*, LNCS 1459:231–256, 1998.
 9. D. Feitelson. Packing Schemes for Gang Scheduling. *Job Scheduling Strategies for Parallel Processing*, LNCS 1162:89–110, 1996.
 10. D. Feitelson. Job Scheduling in Multiprogrammed Parallel Systems. Technical report, IBM T. J. Watson Research Center, 1997. RC 19970 - Second Revision.
 11. D. Feitelson and M. A. Jette. Improved Utilization and Responsiveness with Gang Scheduling. *Job Scheduling Strategies for Parallel Processing*, LNCS 1291:238–261, 1997.
 12. D. Feitelson and L. Rudolph. Distributed Hierarchical Control for Parallel Processing. *IEEE Computer*, pages 65–77, May 1990.
 13. D. Feitelson and L. Rudolph. Mapping and Scheduling in a Shared Parallel Environment Using Distributed Hierarchical Control. In *Proceedings of the 1990 International Conference on Parallel Processing*, 1990.
 14. D. Feitelson and L. Rudolph. Gang Scheduling Performance Benefits for Fine-Grain Synchronization. *Journal of Parallel and Distributed Computing*, 16:306–318, 1992.
 15. D. Feitelson and L. Rudolph. Evaluation of Design Choices for Gang Scheduling Using Distributed Hierarchical Control. *Journal of Parallel and Distributed Computing*, 35:18–34, 1996.
 16. H. Franke, P. Pattnaik, and L. Rudolph. Gang Scheduling For Highly Efficient Distributed Multiprocessor Systems. In *Proceedings of Frontiers'96*, 1996.
 17. A. Hori, H. Tezuka, and Y. Ishikawa. Overhead Analysis of Preemptive Gang Scheduling. *Job Scheduling Strategies for Parallel Processing*, LNCS 1459:217–230, 1998.
 18. M. A. Jette. Performance Characteristics of Gang Scheduling In Multiprogrammed Environments. In *Proceedings of SC'97*, 1997.
 19. B. Kalyanasundaram and K. Pruhs. Speed is as Powerful as Clairvoyance. In *Proceedings of the 36th Symposium on Computer Science*, pages 214–221, 1995.
 20. W. Lee, M. Frank, V. Lee, K. Mackenzie, and L. Rudolph. Implications of I/O for Gang Scheduled Workloads. *Job Scheduling Strategies for Parallel Processing*,

- LNCS 1291:215–237, 1997.
21. W. Leinberger, G. Karypis, and V. Kumar. Job Scheduling in the Presence of Multiple Resources Requirements. In *Proceedings of Supercomputing' 99*, 1999.
 22. W. Leinberger, G. Karypis, and V. Kumar. Multi-Capacity Bin Packing Algorithms with Applications to Job Scheduling under Multiple Constraints. In *Proceedings of the 1999 International Conference On Parallel Processing*, 1999.
 23. M.S. Manasse, L.A. McGeoch, and D.D. Sleator. Competitive Algorithms for On Line problems. In *Proceedings of the Twentieth Annual Symposium on the theory of Computing*, pages 322–333, 1988.
 24. R. Motwani, S. Phillips, and E. Torng. Non-clairvoyant scheduling. *Theoretical Computer Science*, 130(1):17–47, 1994.
 25. J. K. Ousterhout, D. A. Scelza, and P. S. Sindhu. Medusa: An Experiment in Distributed Operating System Structure. *Communications of the ACM*, 23(2):92–105, 1980.
 26. J.K. Ousterhout. Scheduling Techniques for Concurrent Systems. In *Proceedings of the 3rd International Conference on Distributed Comp. Systems*, pages 22–30, 1982.
 27. P.W. Shor. How to do better than best fit: Tight bounds for average case on-line bin packing. In *Proceedings of the 32th IEEE Annual Symposium on Foundations of Computer Science*, pages 752–759, 1990.
 28. F.A.B. Silva and I.D. Scherson. Improvements in Parallel Job Scheduling Using Gang Service. In *Proceedings 1999 International Symposium on Parallel Architectures, Algorithms and Networks*, 1999.
 29. F.A.B. Silva and I.D. Scherson. Towards Flexibility and Scalability in Parallel Job Scheduling. In *Proceedings of the 1999 IASTED Conference on Parallel and Distributed Computing Systems*, 1999.
 30. F.A.B. Silva and I.D. Scherson. Improving Parallel Job Scheduling Using Runtime Measurements. In *Proceedings of the 6th Workshop on Job Scheduling Strategies for Parallel Processing*, 2000.
 31. F.A.B. Silva and I.D. Scherson. Improving Throughput and Utilization on Parallel Machines Through Concurrent Gang. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium 2000*, 2000.
 32. D.D. Sleator and R.E. Tarjan. Amortized Efficiency of List Update and Paging Rules. *Communications of the ACM*, 28(2):202 – 208, 1985.
 33. M. S. Squillante, F. Wang, and M. Papaefthymiou. An Analysis of Gang Scheduling for Multiprogrammed Parallel Computing Environments. In *Proceedings of the 8th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 89–98, 1996.
 34. K. Suzaki and D. Walsh. Scheduling on AP/Linux for Fine and Coarse Grain Parallel Processes. *Job Scheduling Strategies for Parallel Processing*, LNCS 1659:111–128, 1999.
 35. L. G. Valiant. A bridging model for parallel computations. *Communications of the ACM*, 33(8):103 – 111, 1990.
 36. F. Wang, M. Papaefthymiou, and M. S. Squillante. Performance Evaluation of Gang Scheduling for Parallel and Distributed Multiprogramming. *Job Scheduling Strategies for Parallel Processing*, LNCS 1291:277–298, 1997.
 37. L. A. Zadeh. Fuzzy Sets. *Information and Control*, 8:338–353, 1965.